

భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

CANM - 1 PROJECT-REPORT

Prof. Chandra Prakash

1 Objective and understanding of this project

- Learning about analyzing the data using different interpolation methods like Newton divided difference and Chebyshev interpolation
- Finding the derivative of the function we get using numerical differentiation and also the general logic for the chebyshev interpolated polynomial
- Finding the solutions of the polynomial using the Newton-Raphson Method and secant method
- An analysis on the convergence and error of the methods used

2 Importing the Data using Pandas

- The data given is in csv format and can be read using `pandas.read_csv()`.
- It can also be done using file input and output methods, but we used the pandas method because it is easier.

And after the data got imported, we made two lists named x_cord (for the x coordinates) and y_cord (for the y coordinates) and stored the values of x and y coordinates.

3 Python Code for importing the data

```
import pandas as pd
df = pd.read_csv(r"group1.csv")
print(df)
x_cord = df['x'].tolist()
print(x_cord)

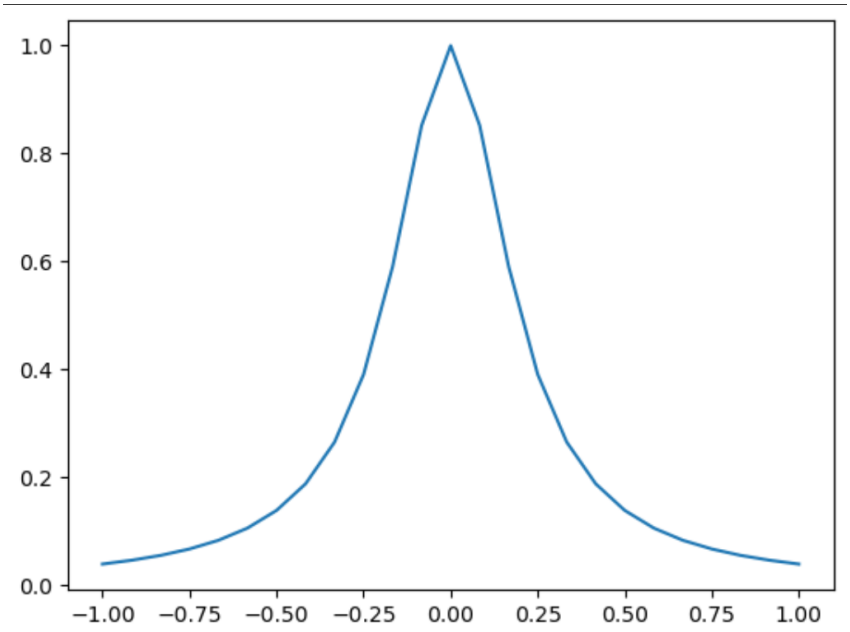
y_cord = df['y'].tolist()
print(y_cord)
```

- These lists are then used as an input for the Newton Divided Difference algorithm.

4 Finding the function that the given values satisfy

We will first plot the given data points to identify what kind of reference function they satisfy.

```
import matplotlib.pyplot as plt
plt.plot(x_cord,y_cord)
```



From what we know this function should be either the plot of a function of type $1/(1+ax^2)$ or it will be a gaussian plot of form e^{-ax^2} generally (This is like hard-coding but was necessary to get the nodes and check if the runge phenomenon still exists for the given dataset)

So let us verify the function, what it belongs to

```
def linspace(a,b,n):
    if n ==1:
        raise ValueError("the denomimator is zero")
    spacing = (b-a)/(n-1)
    return [a + (i*spacing) for i in range(n)]

def check_run(x,y):
    b_a = None
    b_er = float("inf")
    for a in linspace(1, 100, 1000):
        exp = [1/(1+a*(i**2)) for i in x]
        er_ru = sum((y[i]-exp[i])**2 for i in range(len(y)))
        if er_ru < b_er:
            b_er = er_ru
            b_a = a

    return b_a, b_er

a_ru, er_ru = check_run(x_cord, y_cord)
print(a_ru,er_ru)
```

- This gives an output where a is nearly 25 (24.98198) and the error being of order 10^{-7}.
- Had we taken a different interval(like from 24 to 26) we would have better approximation of a value and the error we get will decrease even more drastically.

```
def check_gauss(x,y):
    best_a = None
    best_er = float("inf")
    for a in linspace(1, 100, 1000):
        exc = [2.718 ** ((-i*i)*a) for i in x]
        er_ru = sum((y[i]-exc[i])**2 for i in range(len(y)))
        if er_ru < best_er:
            best_er = er_ru
            best_a = a

    return best_a, best_er

a_g, er_g = check_gauss(x_cord, y_cord)
print(a_g,er_g)
```

- This gives an output where a is nearly 13(13.0900900900900) and the error being of order 10^{-1}(0.12265568836686117).

- So among the two function the function of form $1/(1+ax^2)$ is giving less error when compared to the gaussian function and the a value in that function is nearly equal to 25 which is also the case of Runge function and our Reference function can be approximated to the Runge function.

5 Newton's Divided Difference - Interpolation

Newton's divided difference formula is used to construct an interpolating polynomial for a given set of data points.

The general formula is:

$$f[x_0, x_1, \ldots, x_n] = \frac{f[x_1, \ldots, x_n] - f[x_0, \ldots, x_{n-1}]}{x_n - x_0}$$

The interpolating polynomial is:

$$P(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \ldots$$

6 Data Points

The data points from the csv file are symmetric/equally spaced and defined in the interval $[-1, 1]$. This can be checked by using the max and min functions over the list

7 Code for Newtons Divided Difference - Python

```
# EVALUATING THE COEFFICIENTS OF THE NEWTON INTERPOLATED
   POLYNOMIAL

def newton_div_dif(x, y):
    coef = y[:]
    if len(x) != len(set(x)):
        raise ValueError("THE GIVEN DATASET HAS DUPLICATE VALUES"
           )

    for i in range(1, len(x)):
        for j in range(len(x) - 1, i - 1, -1):
            denominator = x[j] - x[j - i]

            if abs(denominator) < 1e-12:
                raise ZeroDivisionError("POINTS ARE TOO CLOSE")
            coef[j] = (coef[j] - coef[j - 1]) / denominator
    return coef
```

8 Explanation of the code

- The coordinates of y are stored in coef list which is operated on.
- The newtons method doesn't work if there are any same values in x as it will result in division with 0 so we compared the length of the original list with the length of the set formed from that list (as set doesn't have any duplicates).
- Then we started our main part where the logic of the newtons method is implemented with a tolerance of 1e -12 so as that it does not raise and Dividing by Zero floating point error for the difference greater than 1 e-12.
- There are two loops nested to each other the outer loop is for the order of the divided difference and the inner one is for updating the coefficients from the end.
- Here we are following the horners rule and coming from the back.

9 Considerations done while writing the code

- Due to floating-point precision limitations, very small numerical errors may occur when computing denominators of the form:

$$x_j - x_{j-i}$$

- To handle this, a tolerance-based check is used to avoid division by values close to zero.

The output we got was

```
[8]: [0.0384615384615384,
      0.0837439619389763,
      0.1471730036891127,
      0.24776756940875533,
      0.4207521735688538,
      0.7314985618607813,
      1.281740006141349,
      2.075874119068836,
      1.8209487120885246,
      -9.889307678142798,
      -87.66138065918264,
      -256.17594645950265,
      1657.8346280177138,
      2514.1007032553616,
      -33733.17743358906,
      112598.92579584678,
      -211430.70847492988,
      215223.38103675828,
      48767.617900805104,
      -704892.5358257603,
      1718149.6483790255,
      -2869963.152494371,
      3847750.0889006895,
      -4389068.542472266,
      4389068.54247226]
```

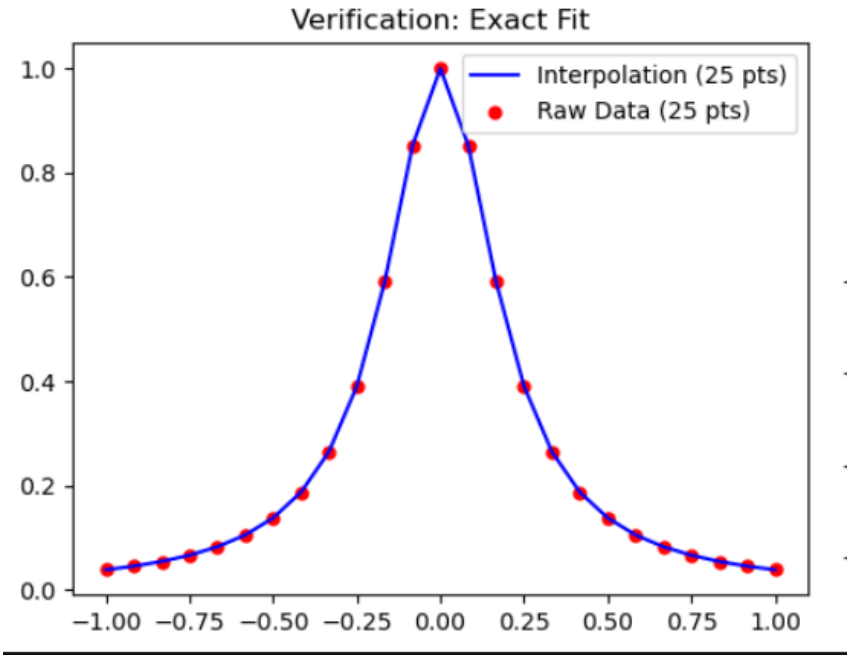
10 The code where we just interpolated the 25 given points with the y values from the newton divided difference is

- This code is a continuation of the previous one,

```
def eval_new(target_x, x_c, coef):
    res = coef[-1]
    for i in range(len(coef) - 2, -1, -1):
        res = res * (target_x - x_c[i]) + coef[i]
    return res

coef_new = newton_div_dif(x_cord, y_cord)
y_new_25 = [eval_new(x, x_cord, coef_new) for x in x_cord]

plt.plot(x_cord, y_new_25, 'b-', label="Interpolation (25 pts)")
plt.scatter(x_cord, y_cord, color='red', s=25, label="Raw Data
   (25 pts)")
```

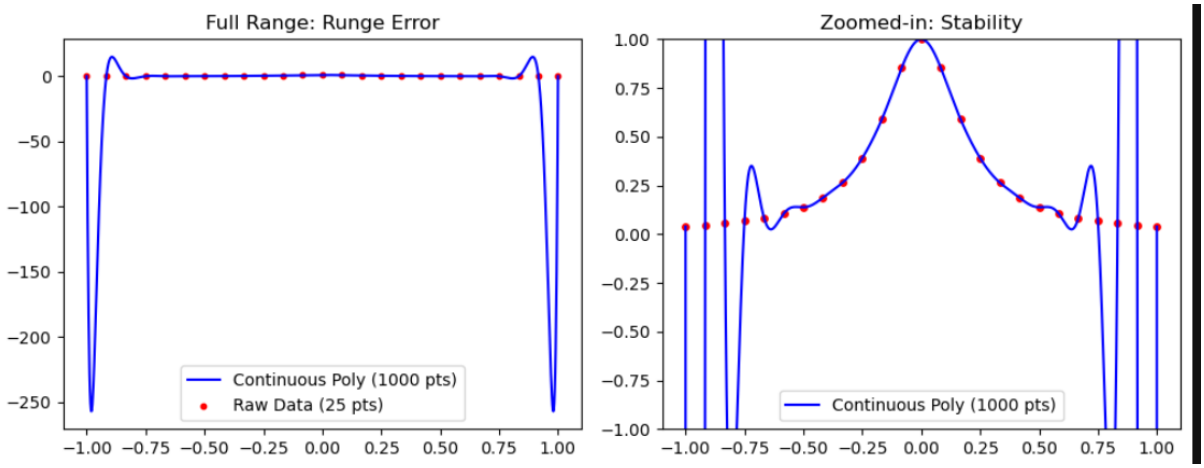


11 The code used for plot when 1000 points are taken to show the Runge phenomenon happening is

This is the continuation part of the code that we used for the coefficients.

```
PTS_T = linspace(-1,1,1000)
x_fin = [i for i in PTS_T]
y_new_1000 = [eval_new(x, x_cord, coef_new) for x in x_fin]

fig, axs = plt.subplots(1, 2)
axs[0].plot(x_fin, y_new_1000, label="Newton Interpolation",
   color="red", linestyle="-")
axs[0].scatter(x_cord, y_cord, label="Data Points (CSV)", color="
   black", s=10)
axs[1].plot(x_fin, y_new_1000, label="Newton Interpolation zoomed
   ", color="red", linestyle="-")
axs[1].scatter(x_cord, y_cord, label="Data Points (CSV)", color="
   black", s=10)
axs[1].set_ylim(-1,1)
axs[0].legend()
axs[1].legend()
```



12 Observations from this interpolation method

- The given dataset does not contain duplicate x-values as there was no error raised from the equality checking line.
- Some y-values repeat due to symmetry, which is valid.
- Newton's Divided Difference method gave us an error? wait what is it? it is the Runge phenomenon.
- Which is the error observed at the edges of the graph where the interpolation blows up because of the oscillations that are forced onto the function for the nodes near to the edges or end points.

13 RUNGE PHENOMENON

- It is a problem of oscillation at the edges of an interval that occurs when using polynomial interpolation with polynomials of high degree over a set of equally spaced interpolation points.
- The magnitude of the n-th order derivatives of this particular function grows quickly when n increases.
- And the other reason is to reduce the lebesgue constant, which has to be minimum for a good interpolant but it grows exponentially if the chebyshev node are not used.

14 The common code part of the Chebyshev interpolation when the nodes are not used when they are used

```
# This is for calculating the chebyshev basis (chebyshev
   polynomials) which are the same wherever
def cheb(x,n):
```

```
    if n ==0:
        return 1
    elif n==1:
        return x
    cur = x
    prev = 1

    for i in range(2,n+1):
        nxt = 2*x*cur - prev
        prev = cur
        cur = nxt

    return cur
# This one is the runge reference function which is used to
   compare the approximation or the interpolation we get with the
    true function plot
def f_runge(x):
    return 1/(1+25*x*x)
# We choose LU Decomposition method because it is faster when
   compared to other methods but one thing is that if the values
   are very less this might not be preffered
# and also we can add the component of the minimization of the
   least squares to get a better accuracy in the roots
def lu_sol(a):
    n = len(a)
    u = list(range(n))

    for i in range(n):
        max_val = -float('inf')

        piv = i
        for j in range(i,n):
          if abs(a[j][i]) > max_val:
                max_val = abs(a[j][i])
                piv = j
        a[i],a[piv] = a[piv],a[i]
        u[i],u[piv] = u[piv],u[i]
        for j in range(i+1,n):
          a[j][i] = a[j][i] / a[i][i]
          for k in range(i + 1, n):
                a[j][k] -= a[j][i] * a[i][k]
    return a,u

def sol(a,p,b):
    n = len(a)
    mid = [b[p[i]] for i in range(n)]
    y = [0.0] * n
    for i in range(n):
        summ = sum(a[i][j] * y[j] for j in range(i))
        y[i] = mid[i] - summ
    x = [0.0] * n
```

```
    for i in range(n - 1, -1, -1):
        summ1 = sum(a[i][j] * x[j] for j in range(i + 1, n))
        x[i] = (y[i] - summ1) / a[i][i]

    return x

# CALCULATING THE CHEBYSHEV APPROXIMATION (Y VALUES)
def cheb_approx(x,coefs):
    summ = 0
    for k in range(len(coefs)):
        summ += coefs[k] * cheb(x, k)
    return summ
```

15 Chebyshev Interpolation/Approximation Where the nodes are not used

Here we did not try to interpolate rather we tried to fit the curve accordingly which doesn't solve the Runge phenomenon as we will still be using the equally spaced points.

```
# Here we are building a matrix of the chebyshev basis functions
lhs_a_ = [[0 for i in range(len(x_cord))] for j in range(len(
   x_cord))]
for i in range(len(x_cord)):
    for j in range(len(x_cord)):
        lhs_a_[i][j] = cheb(x_cord[i],j)


# Now we will solve the equations using the lu-decomposition we
   wrote earlier

LU_matrix_, P_vector_ = lu_sol(lhs_a_)
coefs_cheb = sol(LU_matrix_, P_vector_, y_cord)


print(coefs_cheb)


x_plot_ = PTS_T
y_true_cheb= [f_runge(x) for x in PTS_T]
y_approx_cheb = [cheb_approx(x,coefs_cheb) for x in PTS_T]


# plot them now


fig,axs1 = plt.subplots(1,2,figsize=(18, 6))
# axs1[0].set_ylim(-0.2,1)
axs1[0].plot(x_plot_, y_true_cheb,   label="true",  color="blue")
axs1[0].plot(x_plot_, y_approx_cheb, label="approx", color="red",
    linestyle="--")
```

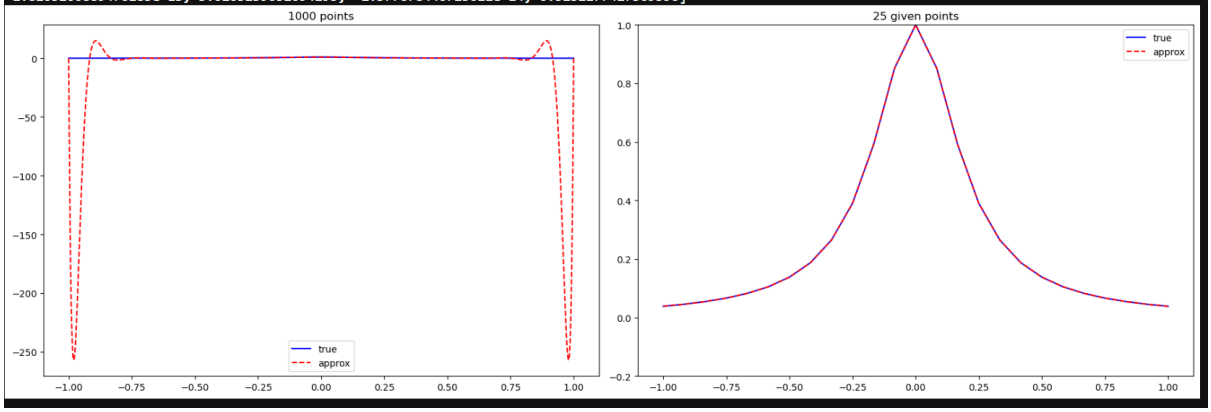
```
axs1[0].legend()
axs1[0].set_title("1000 points")
axs1[1].set_title("25 given points")

y_approx_cheb_1= [cheb_approx(x,coefs_cheb) for x in x_cord]


axs1[1].set_ylim(-0.2,1)
axs1[1].plot(x_cord, y_cord,   label="true",  color="blue")
axs1[1].plot(x_cord, y_approx_cheb_1, label="approx", color="red"
    , linestyle="--")
axs1[1].legend()
plt.tight_layout()
```

- If we are going to use the least squares method, we need to take the transpose of the matrix formed also and then multiply that to the given set and then solve for the coefficients

```
[-33.59492371271685, 2.96489307519158e-12, -62.01049592592349, 2.46724745896867e-12, -45.270685259338116, 1.5636946283269596e-12, -22.28437874872943, 4.2
790841451793117e-13, 2.9585924353000324, -6.708801920999847e-13, 23.8008451013803, -1.4652542915258152e-12, 36.084730360607416, -1.8063599649480487e-12,
37.61060748338302, -1.6142638850481689e-12, 30.53027283667191, -1.1034425704175382e-12, 19.376128893910938, -5.577138796035723e-13, 9.288026365969426, -
1.8205200889476283e-13, 3.0265239652094293, -2.3776784467250l1e-14, 0.5232177427369596]
```



16 Chebyshev Interpolation/Approximation Where the nodes are used

Here we tried to solve the Runge phenomenon using the nodes as they will generally be more concentrated at the corners so it might solve the runge phenomenon.

```
# this function is for calculating the chebyshev nodes because
   they are usually more concentrated near the edges that makes
   them not to generally blow of
# node is generally a point where the Ti(x) (the chebyshev basis
   polynomials become 0)

def cheb_node(n):
    x = linspace(-1,1,1000)
    list1 = []
    for i in range(len(x)-1):
        if cheb(x[i],n)*cheb(x[i+1],n) < 0:
            list1.append((x[i] + x[i+1])/2)
    return list1

nodes = cheb_node(len(x_cord))
y_nodes = [f_runge(x) for x in nodes]




lhs_a = [[0 for i in range(len(nodes))] for j in range(len(nodes)
   )]
for i in range(len(nodes)):
    for j in range(len(nodes)):
        lhs_a[i][j] = cheb(nodes[i],j)



LU_matrix, P_vector = lu_sol(lhs_a)
coefs_nodes = sol(LU_matrix, P_vector, y_nodes)

print(coefs_nodes)

# plot them

x_plot = PTS_T
y_true   = [f_runge(x) for x in x_plot]
y_approx = [cheb_approx(x,coefs_nodes) for x in x_plot]

y_plot_cheb = [cheb_approx(x,coefs_nodes) for x in x_cord]
y_true_cheb = [f_runge(x) for x in x_cord]


fig, axs2 = plt.subplots(1, 2,figsize=(16, 6))
```

```
axs2[0].set_ylim(-0.2,1.2)
axs2[0].plot(x_plot, y_true,   label="true",  color="blue")
axs2[0].plot(x_plot, y_approx, label="approx", color="red",
   linestyle="--")
axs2[0].legend()

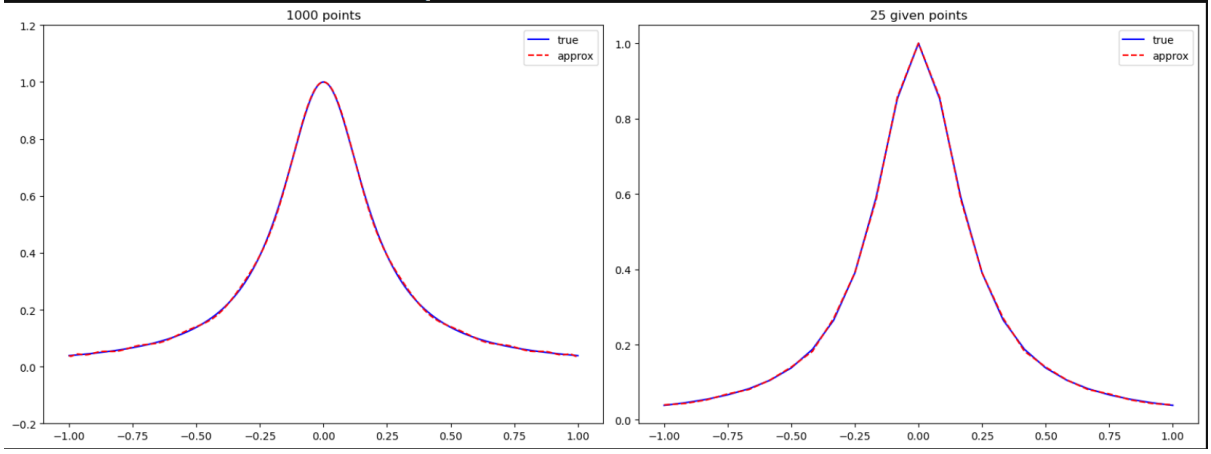

axs2[1].plot(x_cord, y_true_cheb,   label="true",  color="blue")
axs2[1].plot(x_cord, y_plot_cheb, label="approx", color="red",
   linestyle="--")
axs2[1].legend()

axs2[0].set_title("1000 points")
axs2[1].set_title("25 given points")

plt.tight_layout()
plt.show()
```

- If we are going to use the least squares method, we need to take the transpose of the matrix formed also and then multiply that to the given set and then solve for the coefficients

```
[0.19609600195575658, -1.3739995914160436e-17, -0.26379796741071676, 0.0, 0.1771216246194059, 0.0, -0.11927492867522442, -1.732822821832361e-17, 0.080000
84715461334, 6.78267899497801e-18, -0.054075109981459805, -1.1152209243419512e-17, 0.03617849452037732, 1.3267709101591346e-17, -0.02470506907041281, -0.
0, 0.016649836658583914, 1.983832056206901e-17, -0.011752590947916282, 4.170452839561698e-17, 0.008325954817021931, -9.062265152661231e-18, -0.0065223750
9008936, -2.682125710597119e-17, 0.005499199098421526]
```



17 Explanation of the codes

- Then we start making our chebyshev polynomial steps, starting with the cheb(x,n) where n is the number of terms and x is the value that is to be substituted there and this function will return the chebyshev polynomials.
- The given data points are equally spaced, which makes them susceptible to Runge's phenomenon. To demonstrate this, we use the Runge function f(x) = $1/(1+25x^2)$ as a reference, since it is a well-known function that exhibits this phenomenon. We then compare the interpolation results obtained using equally spaced nodes against those obtained using Chebyshev nodes to show how node placement affects stability.
- The method we used to solve the linear system of equations is through the Lu decomposition (we can use other methods also like teh gauss elimination and the QR decomposition)
- Then the function that calculates the y values for us from the coefficients which are multiplied to the chebyshev basis which are calculated above
- THIS COMPLETES THE COMMON PART
- Now the code specific to the part where we did not solve the Runge phenomenon
- here we made a matrix lhs_a_ that initially has all 0 values which is filled by the basis of the chebyshev where we get 25 of them (a 24 degree polynomial)
- Then we got te
- Then we have written a function called lu_sol which basically is the function for solving the system of linear equation using LU decomposition, this function divides A into L and U which are lower triangular matrix and upper triangular matrix to increase the speed of solving the equations.
- The function sol continues after lu_sol which is the solving function for the linear equations and returns two lists which are the lu seperated matrices and the p vector
- And finally the coefficients through the sol function which is the continuation for the lu_sol
- Finally the cheb_approx function which returns a sum of all the terms(the sum of the product of coefficients and Chebyshev polynomials) which was used in making the list y_approx_cheb.
- Now we plot the coordinates that are approximated and the coordinates that are taken as reference using runge function. This was done using the 1000 and the 25 points.
- THIS IS FOR THE PART WHERE THE NODES ARE USED
- The function cheb_node calculates the nodes of the basis or function whatever.
- We are checking for 1000 points where we can find the roots when the product of the 2 consecutive ones are negative as we are not allowed to use the math library and calculate the nodes directly using the cosine inbuilt function.

- The mid points of them were taken considering the closeness to the points where they become 0 but this will have some error while doing so
- Now we follow the similar process as we did for the first part except that the coordinates will be changed to the nodes taken here

18 Observations of the graphs

- The approximation for both the 1000 points and 25 given coordinates closely follows the original function when the nodes were used but in the case where we did not use them it will show runge phenomenon.
- No oscillations are observed near the boundaries for the nodes plot but they were observed for the part where they are not used.
- Hopefully Chebyshev approximation remains stable even for moderate degrees .

19 P'(x) for the Chebyshev polynomial using the logic

We can use the normal differentiation logic in the Chebyshev polynomial and even write the resulting differential function also in that format and then plot the x and y correspondingly. The code used is,

```
def derr_cheby_logic(x,coefs):
    n = len(coefs)
    if n == 1 or n == 0:
        return 0

    elif n == 2:
        return 1

    cp_prev , cp_pre = 1 , x
    cpd_prev , cpd_pre = 0 , 1
    der_sum = coefs[0] * cpd_prev + coefs[1] * cpd_pre
    for i in range(2,len(coefs)):
        cp_curr = 2*x*cp_pre - cp_prev
        cpd_curr = 2*x*cpd_pre + 2*cp_pre - cpd_prev

        der_sum = der_sum + (coefs[i]*cpd_curr)

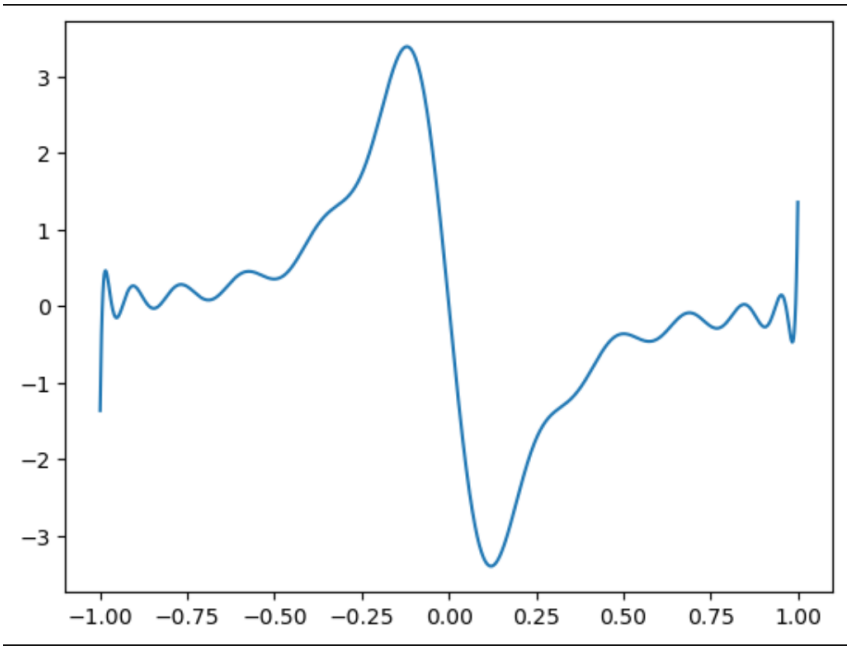
        cp_prev , cp_pre = cp_pre , cp_curr
        cpd_prev , cpd_pre = cpd_pre , cpd_curr

    return der_sum
```

19.1 For 1000 points

```
y_plts = [derr_cheby_logic(x,coefs_nodes) for x in PTS_T]
plt.plot(PTS_T,y_plts)
```

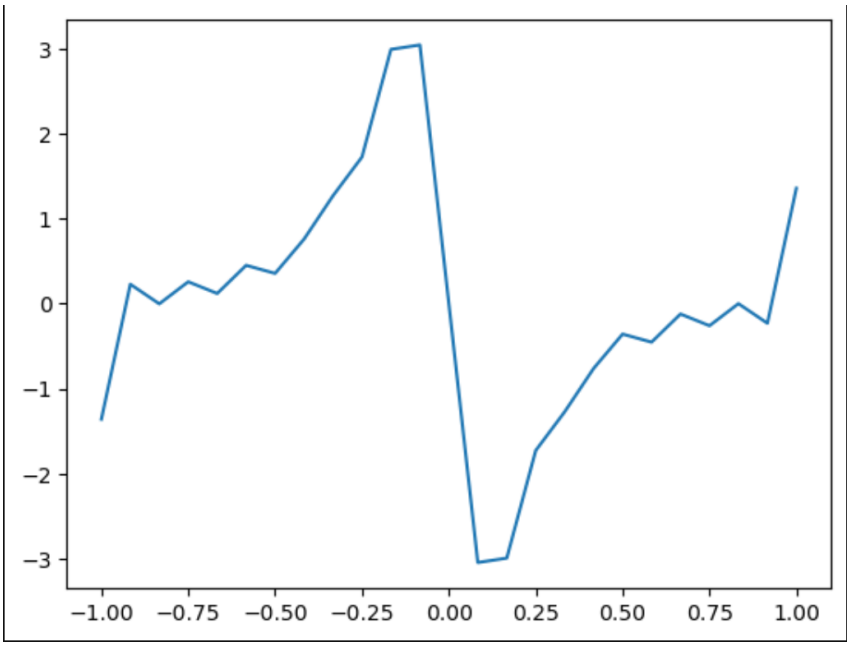
- The corresponding graph will be,



19.2 For 25 points

```
y_plts_25 = [derr_cheby_logic(x,coefs_nodes) for x in x_cord]
plt.plot(x_cord,y_plts_25)
```

- The corresponding graph will be,



- since we are using numerical methods for everything here, we will calculate the derivative by using the numerical differentiation also

20 Numerical Differentiation

Numerical differentiation is used to approximate the derivative of a function using distinct data points.

The most commonly used formulas are:

1. The Forward Difference
2. The Backward Difference
3. The Central Difference

Forward Difference

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

Backward Difference

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}$$

Central Difference

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Higher-order derivatives can also be approximated. For example, the second derivative:

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

It is done by differentiating the first order derivatives.

Smaller values of h generally improve accuracy, but very small h can introduce numerical instability due to floating-point errors.

21 Data Considerations

Numerical differentiation works best when:

- Data points are equally spaced.
- Noise in the data is minimal.
- The function is smooth in the interval under study.

The method we chose to implement here is the central difference.

22 P'(x) for the Chebyshev using numerical differentiation

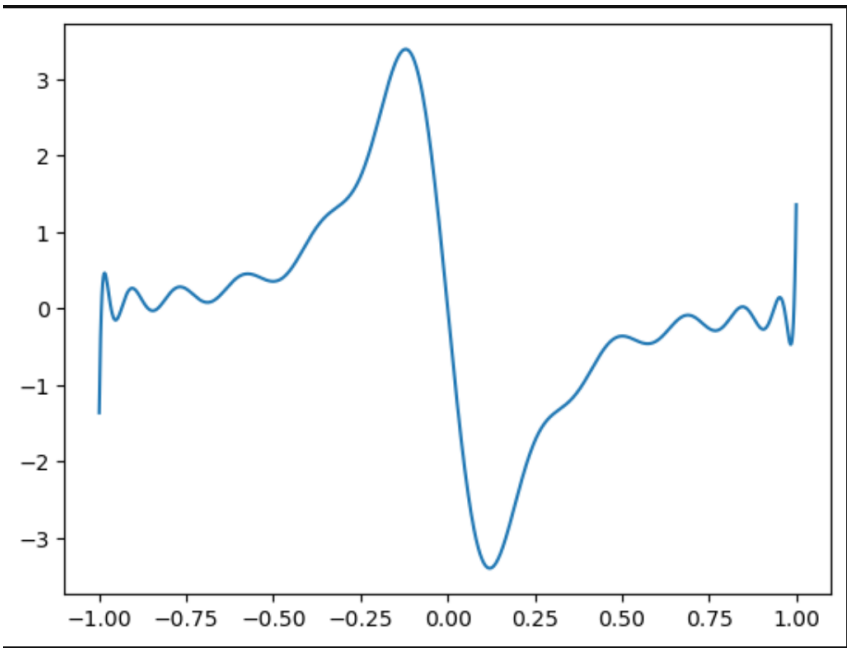
22.1 For 1000 points plot

- This is done by using central difference.

```
def eval_cheby(x):
    return cheb_approx(x,coefs_nodes)

def Numerical_differentiation_central(x,y):
    step = 1e-5
    return (y(x+step) - y(x-step))/(2*step)

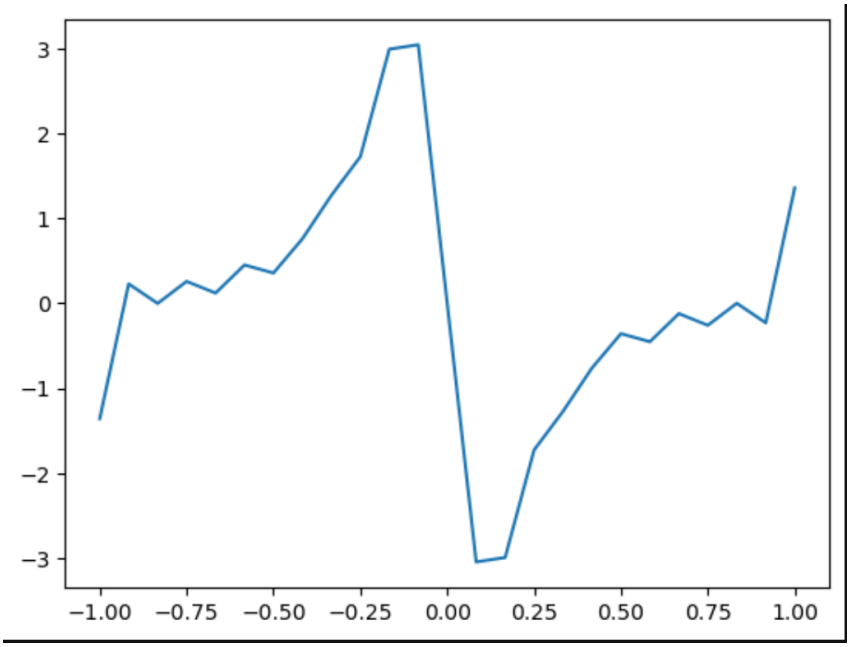
y_nd_ch = [Numerical_differentiation_central(x,eval_cheby) for x
   in PTS_T]
plt.plot(PTS_T,y_nd_ch)
```



22.2 For 25 points plot

- This is done using central difference.

```
y_nd_ch_25 = [Numerical_differentiation_central(x,eval_cheby) for
    x in x_cord]
plt.plot(x_cord,y_nd_ch_25)
```



23 P'(x) for the newton-divided difference polynomial using numerical differentiation

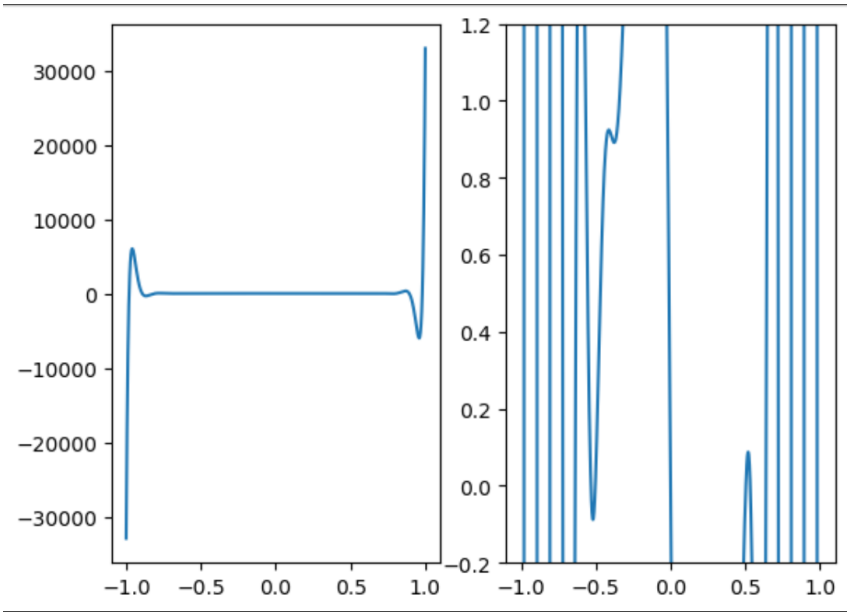
We used the numerical differentiation method only, for the newton-divided difference from the plot we can confirm that there will be runges phenomenon where again the graph blows up at the endpoints and the middle values are seen near to zero and so as to avoid this we can use the chebyshev interpolation method as done above.

23.1 For 1000 points

- This is done using central difference.

```
def newton_approx(x):
    return eval_new(x, x_cord, coef_new)

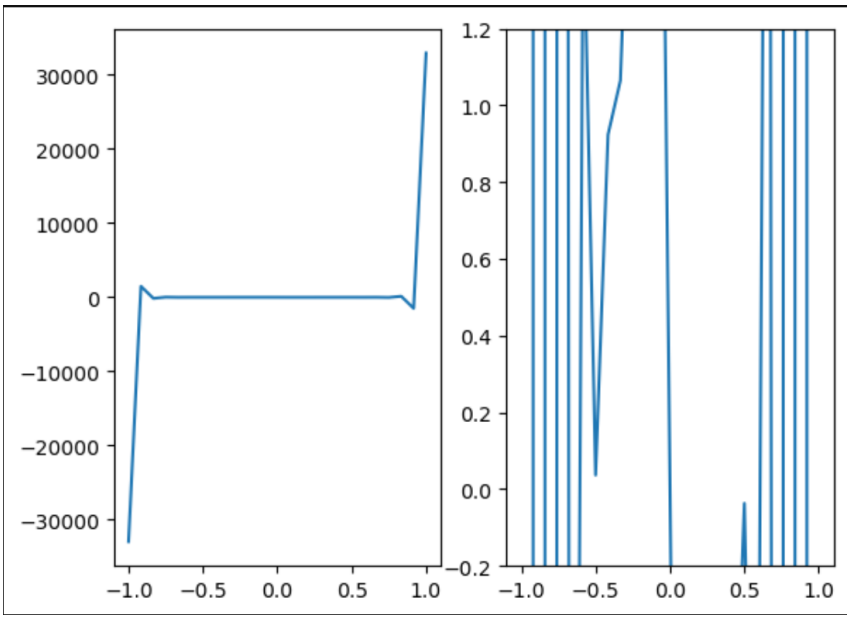
y_new_cord = [Numerical_differentiation_central(x,newton_approx)
   for x in PTS_T]
fig,axs = plt.subplots(1,2)
axs[0].plot(PTS_T,y_new_cord)
axs[1].plot(PTS_T,y_new_cord)
axs[1].set_ylim(-0.2,1.2)
```



23.2 For 25 points

- This is done using central difference.

```
y_new_25cord = [Numerical_differentiation_central(x,newton_approx
    ) for x in x_cord]
fig,axs = plt.subplots(1,2)
axs[0].plot(x_cord,y_new_25cord)
axs[1].plot(x_cord,y_new_25cord)
axs[1].set_ylim(-0.2,1.2)
```



24 The Newton-Raphson Method

It is an iterative numerical technique used to find the roots of a nonlinear equation $f(x) = 0$.

The general iteration formula is:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Starting from an initial guess value we choose x_0, the method generates a sequence of approximations that converge to the root.

25 General Steps

- Choose an initial guess x_0.
- Compute the next approximation using:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

- Repeat until the desired accuracy is achieved:

$$|x_{n+1} - x_n| < \epsilon$$

- The function is smooth and differentiable then this method is highly preferred.
- The convergence of this iterative method mainly depends on the initial value taken if the initial value is close to the root then we can expect a faster quadratic convergence and if the guess was bad it will take ages to converge or it may not converge at all.

26 The code used for Newton-Raphson method for Chebyshev polynomial

```
def Newton_raph(y,g):
    tol = 1e-5
    max_iter = 1000
    x = g
    con_new = []
    for i in range(max_iter):
        fy = y(x)
        dfy = (y(x+tol) - y(x-tol))/(2*tol)
        if abs(dfy) < 1e-15:
            raise ZeroDivisionError("NEARLY A DIVISION WITH 0")
        x_new = x - fy/dfy
        if abs(x_new - x) < tol:
            return x_new,con_new
        con_new.append(abs(x_new-x))
        x = x_new
```

```
    return x,con_new

def cheb_derr(x):
    return Numerical_differentiation_central(x, eval_cheby)

x_tr_n = linspace(-1, 1, 1000)
roots_n = []

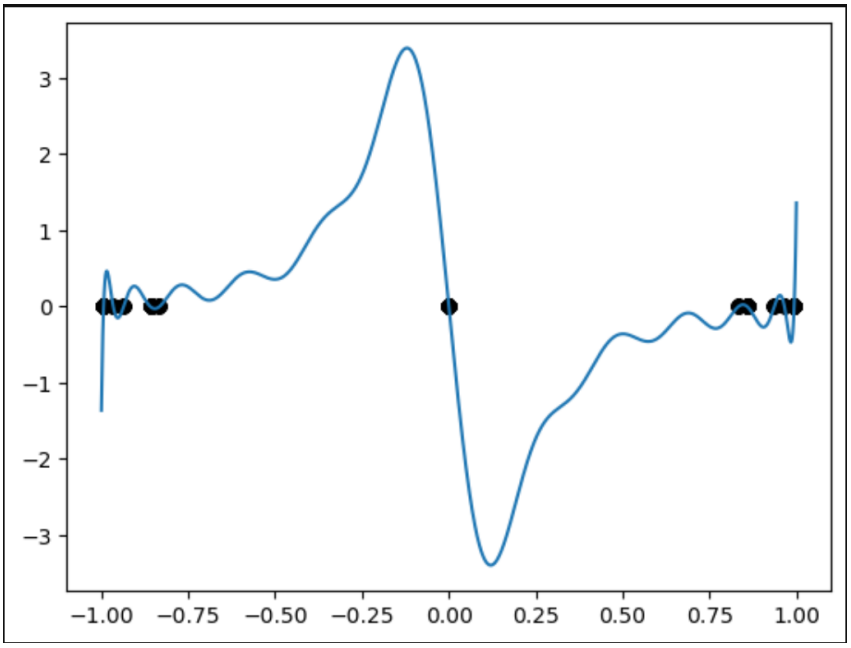
unique_roots_n = []

for i in range(len(x_tr_n)-1) :
    mid_pt = (x_tr_n[i] + x_tr_n[i+1]) / 2
    try:
        r,con_newt = Newton_raph(cheb_derr,mid_pt)
        if -1 <= r <= 1 and abs(cheb_derr(r)) < 1e-5:
            roots_n.append(r)
            if all(abs(r - al_th) > 1e-4 for al_th in
               unique_roots_n):
                unique_roots_n.append(r)
    except ZeroDivisionError:
        pass

# print(roots_n)
# print(len(roots_n))
print(unique_roots_n)
print("")
print("The length of the roots through Newton-Raphson are")
print(len(unique_roots_n))
plt.scatter(roots_n,[cheb_derr(r) for r in roots_n],color = "
   black")
plt.plot(PTS_T,y_plts)
```

27 Resulting roots and the plot of them on the equation

```
[-0.993608136794133, -0.936205884541472, -0.9658136484062193, -0.8330604817347013, -2.623436937254473e-13, -0.858914355363605
9, 0.858914355345468, 0.9362058845414465, 0.9936081358740667, 0.9658136491334615, 0.8330604838996328]
11
```



28 Explanation of the code

- We took a tolerance of 10^{-5}, which is quite decent because even if we take much lower values like 10^{-15}, we might end up with an infinite loop which will effect the convergence and even if the value was taken to be high it will end in the termination of the loop even before all the values are attained.
- The number of iterations were taken to be 1000, through which we can be quite sure that might have covered all the roots possible.
- Then we ran a loop that has our Secant iterative logic which goes through a conditional statement for checking zero division errors, if the value was less that a certain limit.
- Then we wrote a cheb_derr function which computes the first order derivative of the Chebyshev polynomial we got using the numerical differentiation.
- Finding the second order derivative can be done in several ways one of them is commented there in the code and the other was through writing the central difference form again in the code itself.
- Then here we used linspace to get 1000 starting points for the Newton-Raphson method in fact 1000 were chosen because by taking a large set of starting values we can ensure that we found out most of the roots.
- Then we made 2 empty lists namely, the total roots which includes many repeated ones that are obtained by these different starting points from the linspace and the other one as unique roots list
- Then we chose the mid point of the adjacent linspaced values as the starting point.
- Then we applied the Newton-Raphson logic function and segregated the unique roots and all the total ones we get from the loop.

- Then plotted the roots over the graphs to check if they were really the roots.

29 The Secant method

The Secant method is another iterative solver technique just like Newton-Raphson used to find the roots of a nonlinear equation $f(x) = 0$.

One advantage we have for this method is that, unlike Newton-Raphson method which needs the derivative to be know, here we don't require that.

And, there is a disadvantage also that we need to have 2 guesses taken which highly effects the convergences, if anyone of them is not good, then it might take ages to converge.

- The Secant method has superlinear convergence (faster than linear but slower than quadratic).
- It is generally slower than Newton-Raphson but faster than methods like bisection.

The general iteration formula is:

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Starting from two initial guesses x_0 and x_1, the method generates a sequence of approximations that converge to the root.

30 General steps

- Choose two initial guesses x_0 and x_1.
- Compute the next approximation using:

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

- Repeat until the desired accuracy is achieved:

$$|x_{n+1} - x_n| < \epsilon$$

31 Code used for Secant Method for the Chebyshev polynomial

```
def Secant(x1,x2,y):
    tol = 1e-5
    max_iter = 1000
    con_sec = []

    for i in range(max_iter):
        if abs(y(x1) - y(x2)) < 1e-15:
            raise ZeroDivisionError("NEARLY A DIVISION WITH 0")
        x_new = x1 - y(x1) * ((x1 - x2)/(y(x1) - y(x2)))
```

```
        if abs(x_new - x1) < tol:
            return x_new,con_sec
        con_sec.append(abs(x_new - x1))
        x2 = x1
        x1 = x_new


    return x1,con_sec

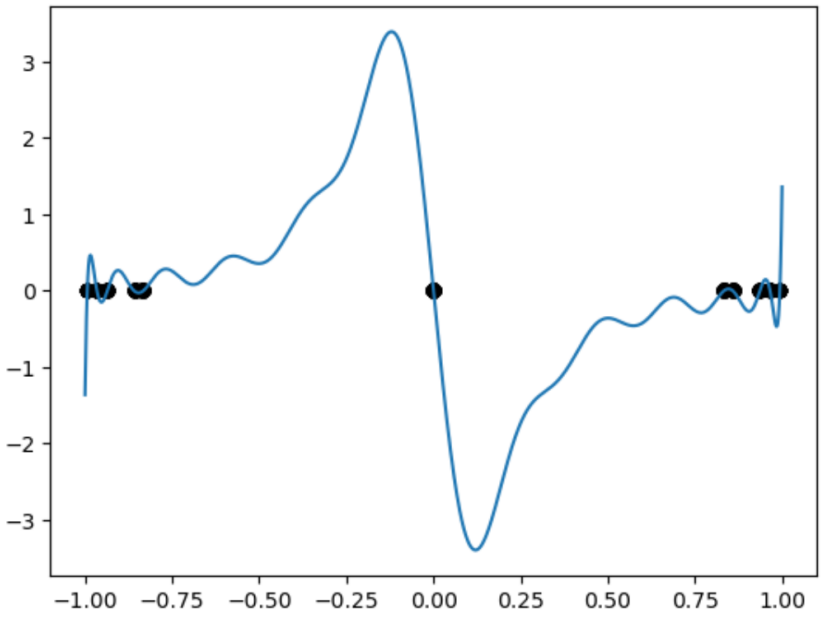


x_chek = linspace(-1, 1, 1000)
roots_s = []
unique_roots_s = []
for i in range(len(x_chek)-1) :
    try:
        r,con_seca = Secant(x_chek[i],x_chek[i+1],cheb_derr)
        if -1 <= r <= 1 and abs(cheb_derr(r)) < 1e-5:
            roots_s.append(r)
            if all(abs(r - al_th) > 1e-4 for al_th in
               unique_roots_s):
                unique_roots_s.append(r)
    except ZeroDivisionError:
        pass

# print(roots_s)
# print(len(roots_s))
print(unique_roots_s)
print("")
print("The lenght of the roots through secant are")
print(len(unique_roots_s))
plt.scatter(roots_s,[cheb_derr(r) for r in roots_s],color = "
   black")
plt.plot(PTS_T,y_plts)
```

32 Resulting roots and plot for the Chebyshev polynomial when secant method is used

```
[-0.9936081614657826, -0.9362058827648339, -0.9658136538084633, -0.833060483105553, -0.8589143534044109, 0.833060481301365, 6.
25837378210441e-10, 0.9362058856306537, 0.8589143517031721, 0.9658136550275207, 0.993608135976948]
11
```



33 Explanation of the code used for secant method

- We took a tolerance of 10^{-5}, which is quite decent because even if we take too much low values like 10^{-15}, we might end up with an infinite loop which will effect the convergence and even if the value was taken to be high it will end in the termination of the loop even before all the values are attained.
- The number of iterations were taken to be 1000 which we can be quite sure that might have covered all the roots possible.
- Then we stored the initial values that are passed as arguments.
- Then we ran a loop that has our secant iterative logic which goes through a conditional statement for checking zero division errors if the value was less that a certain limit.
- Unlike the Newton's method one advantage we have with the secant method is that we don't have to find the derivative of anything.
- Then here we used linspace to get 1000 starting points for the Newton-Raphson method in fact 1000 were cho

34 Observations from the Newton Raphson and secant methods

In fact these is like a comparison between the two methods

- All the unique roots found from the secant and Newton-Raphson method are the (very very close) same.
- The zero root was the only one that got comprehended differently it was because of floating point error in Newton-Raphson it was $6.25 \times 10^{-10} \approx 0$ and in the secant method it was $-2.62 \times 10^{-13} \approx 0$ but both are very close to zero
- So one can be sure that the Newton-Raphson and the secant methods both give nearly the same roots but the convergence depends on the initial values taken in both of them

35 Error and Convergence Analysis

The code that is common for all of the section is

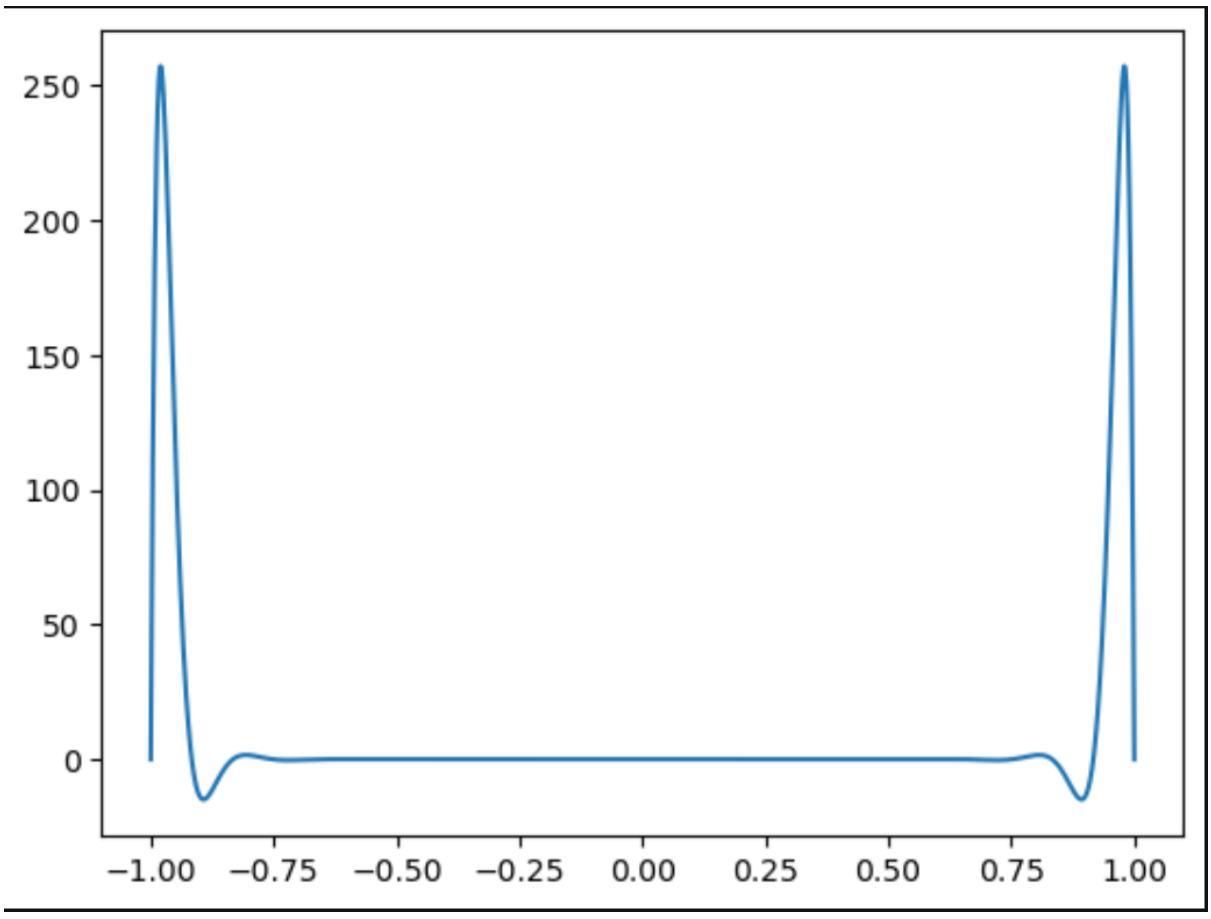
```
f_true = [f_runge(x) for x in PTS_T]
```

35.1 Error analysis for Newton divided difference for 1000 points

The code used here is

```
f_n_d = [eval_new(x,x_cord,coef_new) for x in PTS_T]
f_er_n = [f_true[i] - f_n_d[i] for i in range(len(PTS_T))]
print(abs(max(f_er_n)))
plt.plot(PTS_T,f_er_n)
```

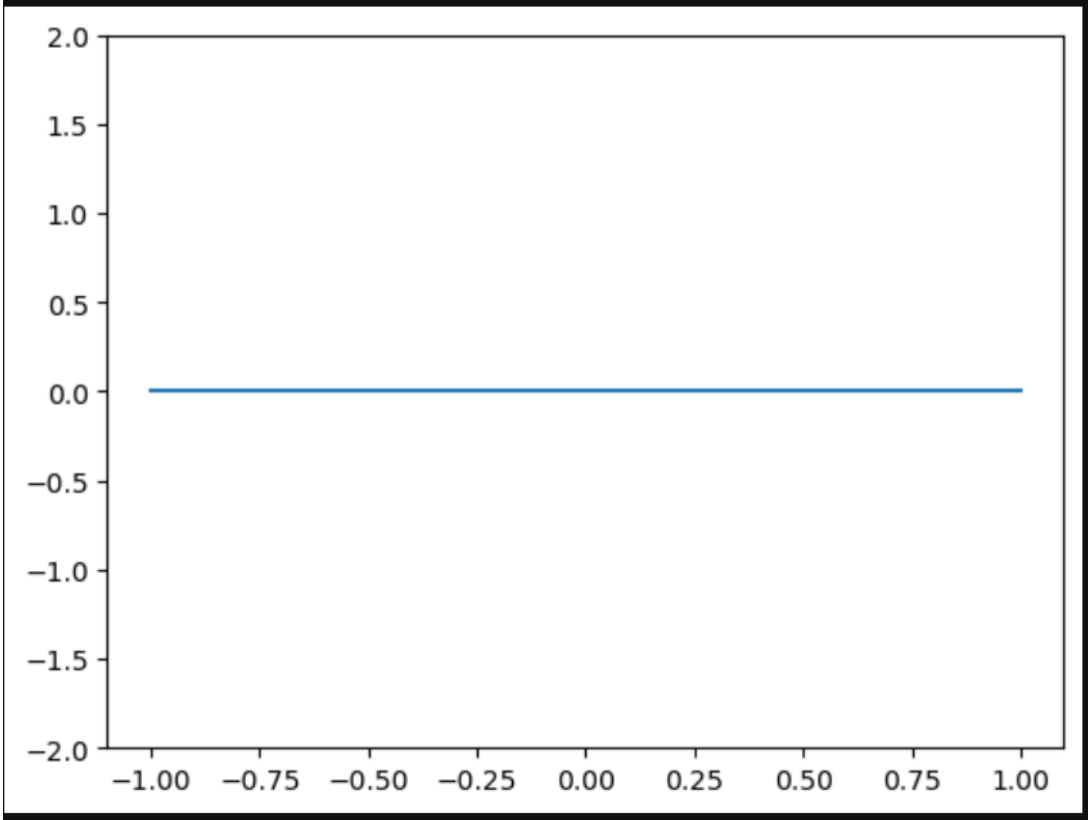
The error plot for this is



The maximum error here is 257.21247481033765

35.2 Error analysis for Newton divided difference for 25 points

```
1.3569700918480976e-13
(-2.0, 2.0)
```



35.3 Error analysis for Chebyshev interpolation

The code used here is

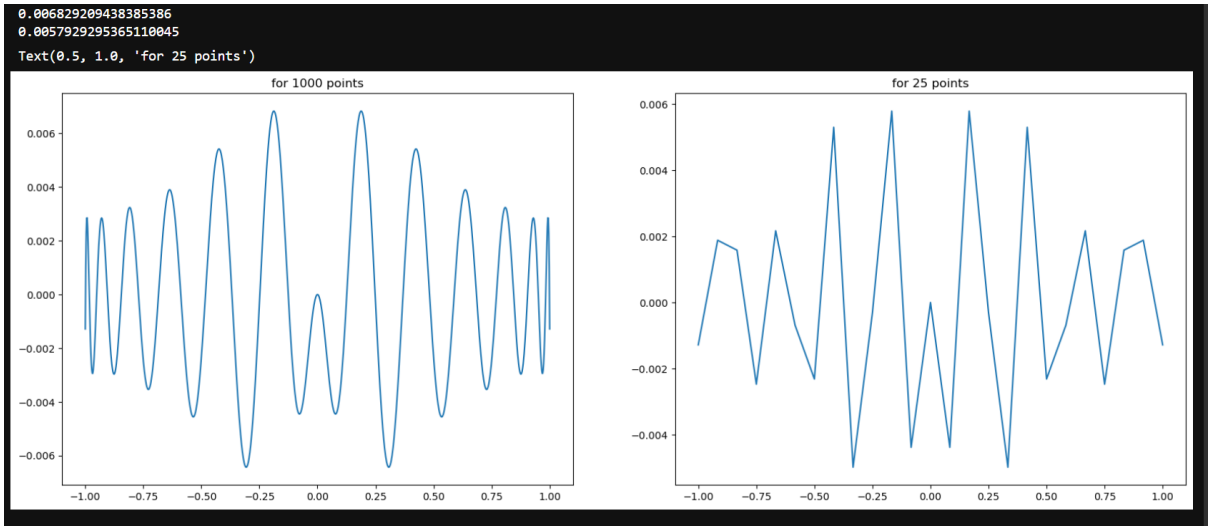
```
f_c = [cheb_approx(x,coefs_nodes) for x in PTS_T]
f_e_c = [f_true[x] - f_c[x] for x in range(len(PTS_T))]

f_c_25 = [cheb_approx(x,coefs_nodes) for x in x_cord]
f_e_c_25 = [f_true_25[x] - f_c_25[x] for x in range(len(x_cord))]

print(abs(max(f_e_c)))

print(abs(max(f_e_c_25)))

fig,axs8 = plt.subplots(1, 2,figsize=(20, 7))
axs8[0].plot(PTS_T,f_e_c)
axs8[0].set_title("for 1000 points")
axs8[1].plot(x_cord,f_e_c_25)
axs8[1].set_title("for 25 points")
```



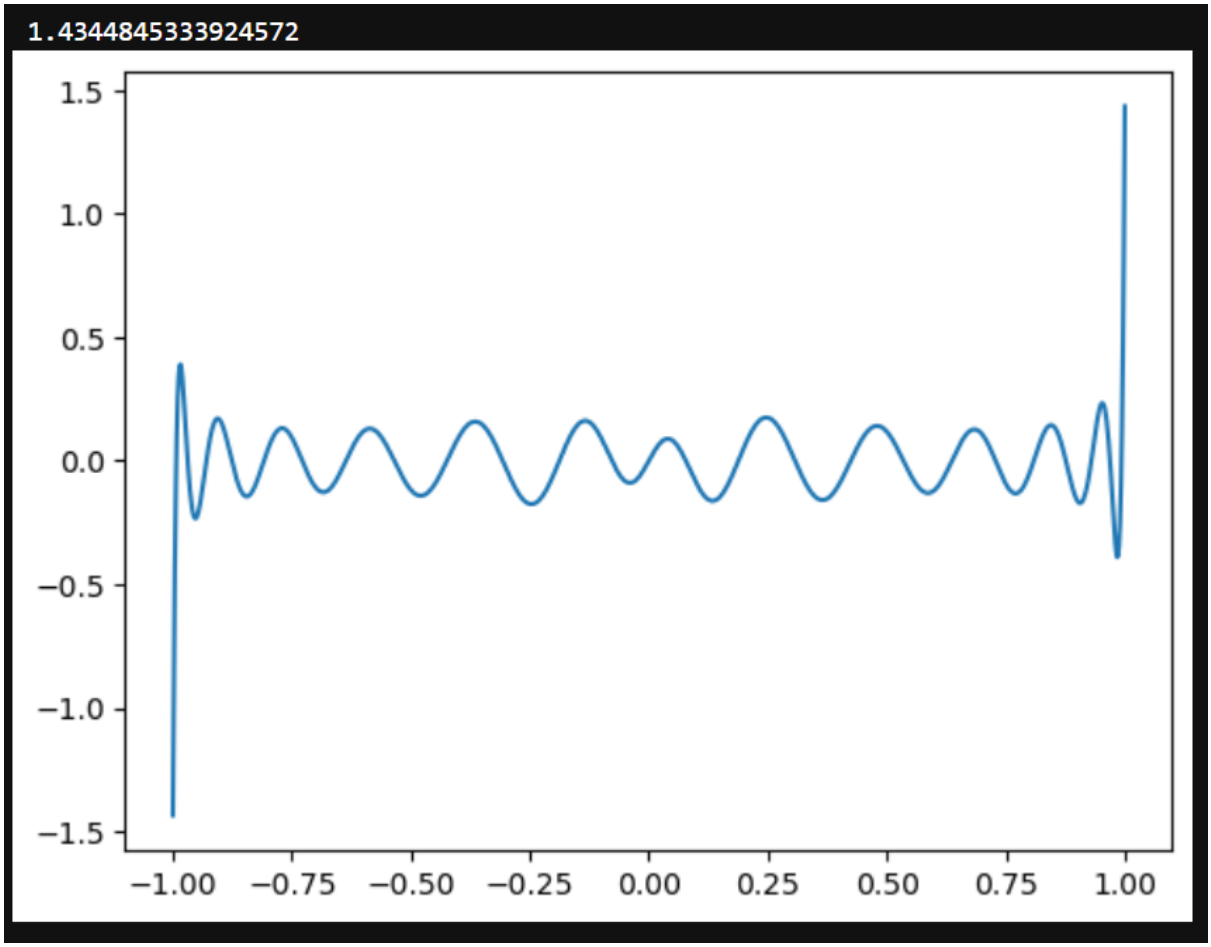
The maximum error here is 0.006829209438385386

35.4 Error analysis for Numerical Differentiation

The code used here is for all the 3 types of errors

```
f_t_d= [-50*x/(1+25*(x**2))**2 for x in PTS_T]
f_d_c = [Numerical_differentiation_central(x,eval_cheby) for x in
    PTS_T]

f_e_d_c = [f_d_c[i] - f_t_d[i] for i in range(len(PTS_T))]
plt.plot(PTS_T,f_e_d_c)
print(max(f_e_d_c))
```



35.5 Error and convergence analysis

The reference function taken here is the bisection method one, we chose bisection method because it has the best accuracy while finding the roots but one problem is that it is slow. The code used here for bisection methods is

```
def bisection_method(f,a,b):
    tolerance_b = 1e-12
    max_iterations = 1000
    con_bis = []

    if (f(a) * f(b)) > 0 :
        raise ValueError("this set of points are both positibe or
            both negative")

    for i in range(max_iterations):
        mid_pt = (a + b) / 2
        con_bis.append(abs((b-a)/2))

        if abs(f(mid_pt)) < tolerance_b or (b-a) / 2 <
           tolerance_b:
            return mid_pt,con_bis

        if f(a) * f(mid_pt) < 0:
```

```
            b = mid_pt
        else:
            a = mid_pt

    return a,con_bis


roots_b = []

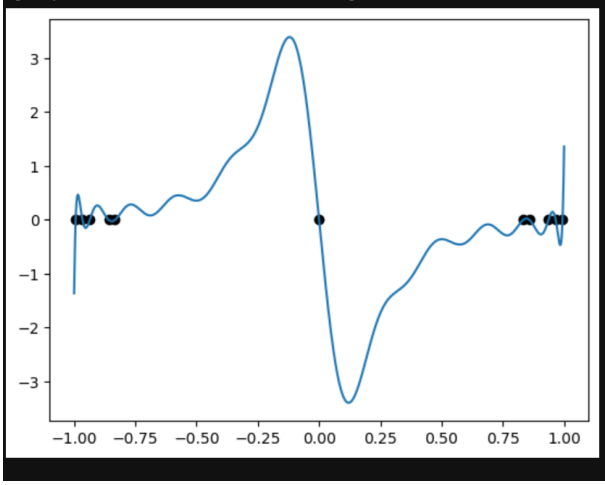
for i in range(len(PTS_T) - 1):
    try:
        r,con_bise = bisection_method(cheb_derr, PTS_T[i], PTS_T[
            i+1])
        if -1 <= r <= 1 and abs(cheb_derr(r)) < 1e-6:

            if all(abs(r - u) > 1e-4 for u in roots_b):
                roots_b.append(r)
    except ValueError:
        pass

roots_b.sort()
print(len(roots_b))
print(roots_b)
plt.scatter(roots_b,[cheb_derr(r) for r in roots_b],color = "
    black")
plt.plot(PTS_T,y_plts)
```

The roots and the plot when this method is used are given below

```
11
[-0.9936081358111343, -0.9658136471529845, -0.9362058845414846, -0.858914355335345, -0.8330604839432347, 5.551115123125783e-17, 0.833060483944167, 0.8589
14355335345, 0.9362058845414846, 0.9658136471529845, 0.993608135811134]
[<matplotlib.lines.Line2D at 0x1e1db125a90>]
```



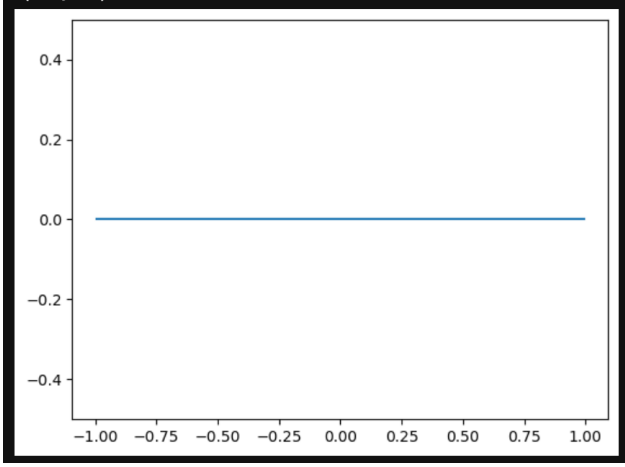
35.6 Error analysis for Newton-Raphson

```
roots_b.sort()
unique_roots_n.sort()
f_error_new = [roots_b[i] - unique_roots_n[i] for i in range(len(
    roots_b))]
```

```
print(f_error_new)

# fig,axs10 = plt.subplots(1, 2,figsize=(20,11))
plt.plot(unique_roots_n,f_error_new)
plt.ylim(-0.5,0.5)
# axs10[1].plot(unique_roots_n,f_error_new)
```

```
[9.829986824527737e-10, 1.2532348492300116e-09, -1.2545520178264269e-14, 2.8260949136438285e-11, -2.2085334672183876e-09, 2.6239920487667856e-13, 4.45341
5414218398e-11, -1.0123013538532177e-11, 3.808064974464287e-14, -1.980477004615011e-09, -6.293265908396961e-11]
(-0.5, 0.5)
```

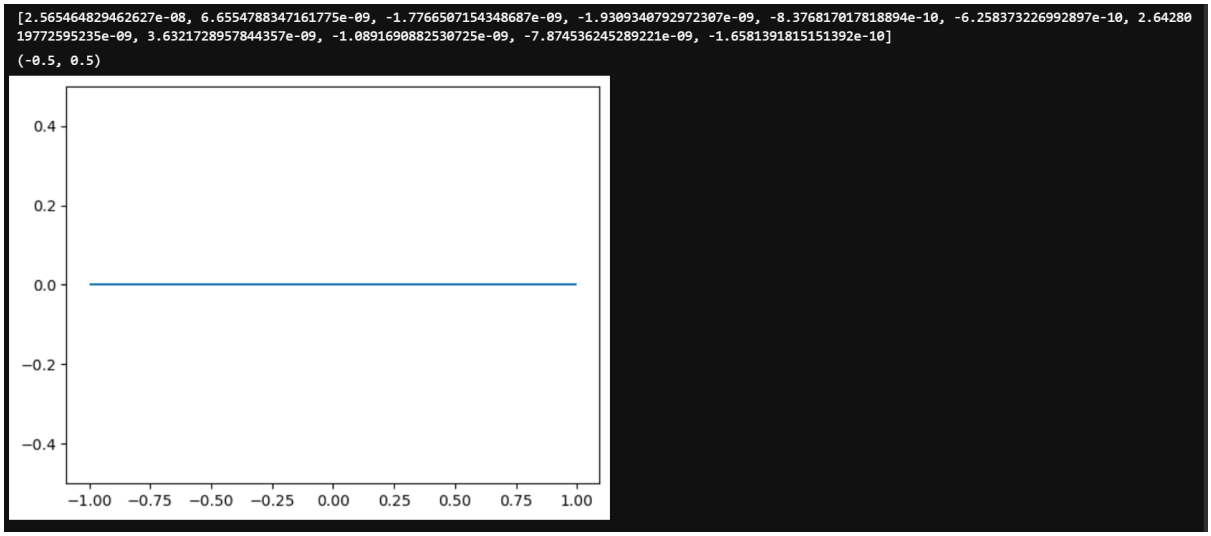


35.7 Error analysis for Secant

The code used here is

```
roots_b.sort()
unique_roots_s.sort()
f_error_se = [roots_b[i] - unique_roots_s[i] for i in range(len(
    roots_b))]
print(f_error_se)


# fig,axs11 = plt.subplots(1, 2,figsize=(20,11))
plt.plot(unique_roots_s,f_error_se)
plt.ylim(-0.5,0.5)
# axs11[1].plot(unique_roots_s,f_error_se)
```



35.8 Convergence analysis for Newton-Rapshon and Secant method

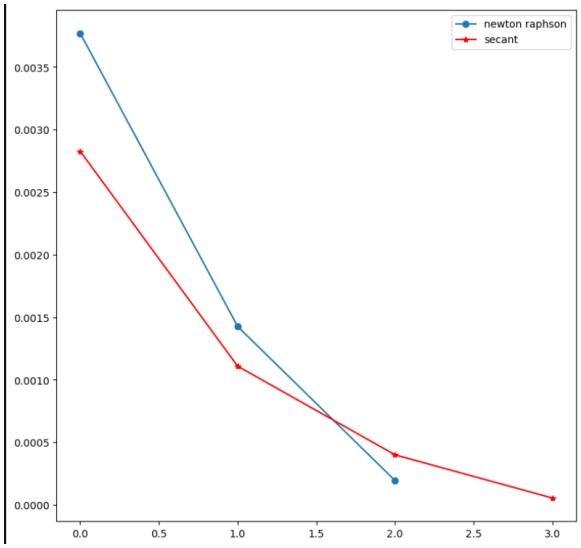
```
fig,axs12 = plt.subplots(1, 2,figsize=(20,9))

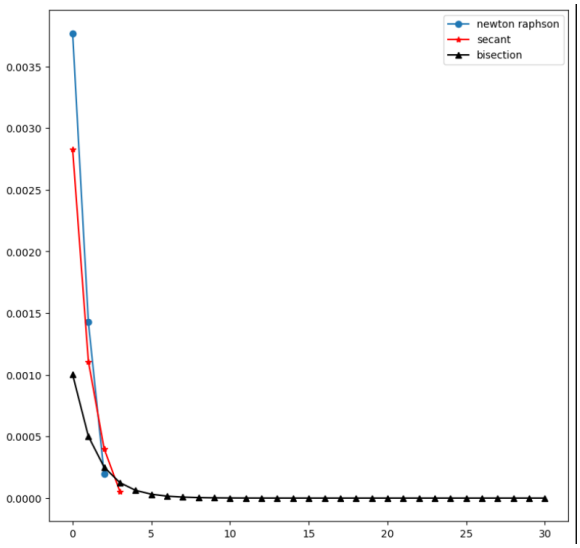
axs12[0].plot(con_newt,label = "newton raphson", marker ='o')
axs12[0].plot(con_seca,label = "secant",marker = '*',color = "red
    ")


axs12[1].plot(con_newt,label = "newton raphson", marker ='o')
axs12[1].plot(con_seca,label = "secant",marker = '*',color = "red
    ")
axs12[1].plot(con_bise,label = "bisection", marker = '^',color =
    "black")

axs12[0].legend()
axs12[1].legend()

plt.show()
```


newton raphson
secant
0.0035
0.0030
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
0.0
0.5
1.0
1.5
2.0
2.5
3.0


newton raphson
secant
bisection
0.0035
0.0030
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
0
5
10
15
20
25
30

Team Members

1. Rayanki Lakshmi Navneeth (CO25BTECH11022)
2. Ganta Tulasi Bharadwaj (CO25BTECH11008)
3. Akkim Dheeraj Siddhartha (CO25BTECH11002)
4. Nakirikanti Venkata Rajiv Varshith (CO25BTECH11016)
5. Karri Siva Lochan (CO25BTECH11012)

THE END